# Coupling Statutory Rules and Administrative Processes: A Pragmatic Contract-Based Approach to Rules-as-Code

**Author:** Dylan A Mordaunt
**ORCID:** 0000-0002-9775-0603
**Affiliations:**

1. Faculty of Health, Education and Psychology, Victoria University of Wellington
2. College of Medicine and Public Health, Flinders University
3. Centre for Health Policy, The University of Melbourne

**Date:** July 2026
**Track:** `community_20260704` (Phase 4)
**arXiv status:** submission candidate v0.3.0; authorization pending (see `ARXIV_SUBMISSION.md`; GitHub #15) **Author block:** `papers/AUTHOR.md`

---

## Abstract

Rules-as-Code (RaC) initiatives often treat rules-heavy calculations and process-heavy administration separately, although operational systems must connect them. We describe a contract-based boundary between deterministic rules modules and administrative process engines. The boundary combines versioned parameters, provenance-labelled fixtures, value-state semantics, and calculation traces while leaving discretionary judgments with human decision-makers. We evaluate the approach through a New Zealand Official Information Act (OIA) clock module and a separate cross-engine study of US Supplemental Nutrition Assistance Program (SNAP) calculations. Repository tests establish import-graph isolation for the OIA module and agreement for 50 approved SNAP comparison cases; 15 additional SNAP cases remain separately classified because their model surfaces or assumptions are not directly comparable. These results support a narrower claim than universal interchange: small, versioned contracts can make selected rule/process integrations testable and auditable without requiring participating systems to share an execution language.

---

## 1. Introduction

Public benefits and administrative processes are governed by statutory guidelines. Standard software engineering practices often lead to "hidden rules" where statutory parameters (such as timelines, limits, and allowances) are tightly coupled with application-level database states, UI controllers, and process routing.

To address this, we propose a decoupled architecture using **Policy Interchange Contracts (PIC)**. PIC defines standard JSON formats for:

1. **Parameters** (`pic-parameters`): Versioned thresholds and calendar exclusions.
2. **Fixtures** (`pic-fixtures`): Human-curated test scenarios with clear provenance.
3. **Traces** (`pic-traces`): Step-by-step audit logs mapping executions to statutory sections.

---

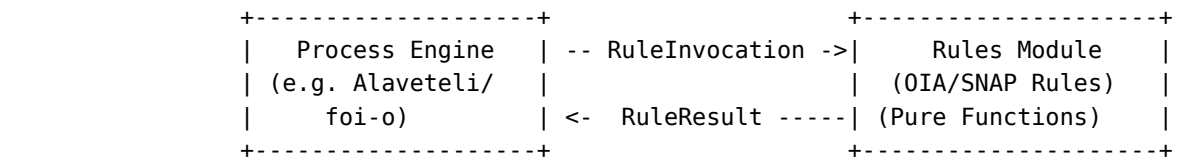
## 2. Methodology & Architecture

The architecture and evidence surfaces are summarized in `FIGURE_1_ARCHITECTURE.md` and `ARTIFACT_SUMMARY.md`.

### 2.1 The Invocation Seam

We decouple rules from process engines by establishing a typed, message-based seam:

- **`RuleInvocation`**: Exposes a target decision ID, a parameter set version, and a dictionary of inputs wrapped as `ValueObjects` (carrying `value` and `valueState`).
- **`RuleResult`**: Exposes the computed outputs, any validation warnings, and optional execution trace steps.
- **`DiscretionPoint`**: Emitted when a statutory decision requires human evaluation (e.g. assessing whether an extension reason is "reasonable" or if "urgency" is justified), preventing machine auto-certification of discretionary parameters.




### 2.2 Domain Separation

- **Rules-Heavy Domain (SNAP)**: Tested using the `policyengine-us` and PRD runners over five states.
- **Process-Heavy Domain (OIA)**: Tested using the OIA rules module staged under `foi-o`.

---

## 3. Related Work

Existing legal-rule formalisms often attempt to capture complete statutory semantics in heavy semantic webs or domain-specific languages:

- **L4 and Catala**: Provide rigorous, executable statutory logic but require compilation pipelines and have steep learning curves for administrative developers.
- **LegalRuleML & Akoma Ntoso**: Provide XML-based markup schemas for legal documents but do not easily integrate with active web application runtimes (like Ruby on Rails or Play Framework).

Our approach uses standard JSON/YAML schemas, allowing easy adoption in existing benefit and request systems.

---

## 4. Empirical Evaluation & Claims

Our repository artifacts support the following verification claims:

| Claim | Supporting Repo Artifact | Verification Result |
|---|---|---|
| OIA rules are fully isolated from process states | `external/foi-o/tests/test_oia_rules.py` | Import-isolation test confirms zero process imports in the rules module. |
| Decoupled rules module matches retained reference cases | `external/foi-o/tests/test_oia_rules.py` | All 13 reviewed clock cases pass the differential regression test. |
| Multi-engine SNAP calculations can be compared | `studies/snap-divergence/results/comparison-approved-results.jsonl` | 50 approved comparison cases agree within the declared tolerance. |
| Divergences can be systematically classified | `studies/snap-divergence/DIVERGENCE_CLASSIFICATION.md` | All 15 divergences classified under state-option, vintage, or triggers. |

## Data and code availability

All code, schemas, test inputs, runner implementations, result artifacts, and adjudication documentation used for these claims are versioned in this repository. Engine and source revisions are recorded in the relevant study manifests and reports. External services are not required for the deterministic repository validation suite, although reproducing live engine runs requires the pinned third-party engines.

## Limitations

The demonstrations cover selected OIA deadline calculations and selected SNAP scenarios; they do not establish complete statutory coverage, legal correctness, population representativeness, or interoperability among arbitrary engines. Agreement between two implementations is not an independent legal oracle. The OIA reference cases depend on recorded human interpretation, while the SNAP held cases show that adapter assumptions and model scope can prevent direct comparison. Upstream adoption is also incomplete: several related contributions remain under maintainer review.

## 5. Conclusion

The evidence supports a practical architectural result: selected statutory calculations can be isolated behind typed, versioned contracts while process state and discretionary decisions remain outside the calculation module. Differential testing then exposes both agreements and comparability limits. Further work should test additional consumers and policy domains before treating PIC as a general interoperability standard.